Search files

incident_agent
static
tests
app.py
article.md
article.md
run_benchmark.py
run_simulation.py


```python
from __future__ import annotations
import os
import asyncio
from typing import Optional, Dict, Any, List
from fastapi import FastAPI, HTTPException
from fastapi.staticfiles import StaticFiles
from fastapi.responses import FileResponse, JSONResponse
from pydantic import BaseModel

from incident_agent.models import Alert
from incident_agent.orchestrator import IncidentOrchestrator, SHARED_INFRA_BANK
from incident_agent.simulator import generate_database_failover_cascade, generate_mixed_storm
from incident_agent.benchmark import run_comparative_benchmark

app = FastAPI(title="Hindsight Incident Agent", version="1.0.0")

# Shared orchestrator instance
orchestrator = IncidentOrchestrator()

# Mount static files
app.mount("/static", StaticFiles(directory="/workspace/static"), name="static")

@app.get("/")
async def root():
    return FileResponse("/workspace/static/index.html")

@app.get("/api/health")
async def health():
    return {"status": "ok", "engine": "hindsight-coalesced-memory"}

class SimulationRequest(BaseModel):
    scenario: str = "database"  # database, redis, kafka, mixed
    count: int = 40
    use_coalescing: bool = True

@app.post("/api/simulate")
async def simulate(req: SimulationRequest):
    if req.scenario == "mixed":
        alerts = generate_mixed_storm(num_database=int(req.count * 0.6), num_redis=int(req.count * 0.25), n
    elif req.scenario == "redis":
        from incident_agent.simulator import SERVICES
        alerts = [
            Alert(
```

```
async def simulate(req: SimulationRequest):
    elif req.scenario == "redis":
        alerts = [
            Alert(
                id=f"alt-redis-{i+1:03d}",
                service=SERVICES[i % len(SERVICES)],
                message=f"RedisCommandTimeout: command GET session_{i*13} timed out on auth-cache.internal
                severity="warning",
                labels={"component": "redis", "alertname": "redis_pool_starvation", "target": "auth_cache
            )
            for i in range(req.count)
        ]
    elif req.scenario == "kafka":
        from incident_agent.simulator import SERVICES
        alerts = [
            Alert(
                id=f"alt-kafka-{i+1:03d}",
                service=SERVICES[i % len(SERVICES)],
                message=f"ConsumerLagSpike: consumer group order-processors lag exceeds 45000 msgs on par
                severity="warning",
                labels={"component": "kafka", "alertname": "consumer_lag_spike", "target": "order_events"
            )
            for i in range(req.count)
        ]
    else:  # database
        alerts = generate_database_failover_cascade(num_alerts=req.count)

    results, metrics = await orchestrator.process_storm(alerts, use_coalescing=req.use_coalescing)

    return {
        "metrics": metrics.model_dump(),
        "triage_stream": [r.model_dump() for r in results],
        "top_runbook": results[0].suggested_runbook if results else None,
        "sample_alerts": [a.model_dump() for a in alerts[:6]]
    }

class BenchmarkRequest(BaseModel):
    num_alerts: int = 50

@app.post("/api/benchmark")
async def benchmark(req: BenchmarkRequest):
    alerts = generate_mixed_storm(
        num_database=int(req.num_alerts * 0.6),
        num_redis=int(req.num_alerts * 0.25),
```

```python
async def benchmark(req: BenchmarkRequest):
    alerts = generate_mixed_storm(
        num_redis=int(req.num_alerts * 0.25),
        num_kafka=int(req.num_alerts * 0.15)
    )
    data = await run_comparative_benchmark(alerts)
    return data

@app.get("/api/banks")
async def get_banks():
    """Inspect retained records in Hindsight memory banks."""
    local_banks = orchestrator.adapter._local_banks
    output = []
    for b_id, bank in local_banks.items():
        output.append({
            "bank_id": b_id,
            "record_count": len(bank.records),
            "records": [
                {
                    "id": r["id"],
                    "content": r["content"],
                    "metadata": r.get("metadata", {}),
                    "timestamp": r["timestamp"]
                }
                for r in bank.records
            ]
        })
    return {"banks": output}

class SingleAlertRequest(BaseModel):
    service: str
    message: str
    component: Optional[str] = None
    alertname: Optional[str] = None

@app.post("/api/triage")
async def triage_single(req: SingleAlertRequest):
    alert = Alert(
        id="alt-custom-001",
        service=req.service,
        message=req.message,
        labels={
            "component": req.component or req.service,
            "alertname": req.alertname or "custom_incident"
```

```python
class SingleAlertRequest(BaseModel):
    service: str
    message: str
    component: Optional[str] = None
    alertname: Optional[str] = None

@app.post("/api/triage")
async def triage_single(req: SingleAlertRequest):
    alert = Alert(
        id="alt-custom-001",
        service=req.service,
        message=req.message,
        labels={
            "component": req.component or req.service,
            "alertname": req.alertname or "custom_incident"
        }
    )
    result = await orchestrator.triage_alert(alert, use_coalescing
    return result.model_dump()

```

SRE OPS **Hindsight Incident Agent** Concurrent Memory Engine

Hindsight Memory: Active | GitHub | Docs

Simulate Alert Cascade

Inject concurrent alerts into memory triage pipeline

FAILURE DOMAIN SCENARIO

PostgreSQL Failover Cascade (Primary pool exhausted)

ALERT VOLUME

40 concurrent alerts

QUERY OPTIMIZATION

Coalesced (Single-Flight)

⚡ Trigger Alert Cascade

Run Comparative Benchmark

Cascade Performance HUD

Single-Flight Coalesced

INGESTED ALERTS	UNIQUE FINGERPRINTS	MEMORY OUTBOUND CALLS
0	0	0
Parallel microservice events	Structural signatures	Actual network fetches

CALLS COALESCED (ZERO I/O)	P99 TRIAGE LATENCY	RETAINED RECORDS
0	0 ms	0
0% duplicate queries saved	Sub-second recovery goal	Deduplicated in Hindsight

Live Alert Triage Stream

0 alerts processed

ALERT ID	SERVICE	FINGERPRINT HASH	COMPONENT	QUERY MODE	CONFIDENCE	LATENCY

Trigger an alert cascade above to stream triage events.

Synthesized Incident Runbook

Recalled and assembled from Hindsight Memory Bank

No active incident runbook generated yet. Trigger a cascade to view

SRE OPS **Hindsight Incident Agent** Concurrent Memory Engine

Hindsight Memory: Active GitHub Docs

Simulate Alert Cascade

Inject concurrent alerts into memory triage pipeline

FAILURE DOMAIN SCENARIO

Mixed Infrastructure Storm (Multi-tier outage)

ALERT VOLUME

60 concurrent alerts

QUERY OPTIMIZATION

Naive (Uncoordinated Fan-Out)

⚡ Trigger Alert Cascade

Run Comparative Benchmark

Cascade Performance HUD

Naive Fan-Out

INGESTED ALERTS	UNIQUE FINGERPRINTS	MEMORY OUTBOUND CALLS
60	14	60
Parallel microservice events	Structural signatures	Actual network fetches

CALLS COALESCED (ZERO I/O)	P99 TRIAGE LATENCY	RETAINED RECORDS
0	67.5 ms	14
0% duplicate queries saved	Sub-second recovery goal	Deduplicated in Hindsight

Benchmark Results: Naive Fan-Out vs. Single-Flight Coalescing

×

Naive Execution (Uncoordinated)

Outbound Calls:	60 calls
P99 Latency:	66.9 ms
Coalesced:	0 calls
Retained Records:	14 records
Cache Stampede:	High Contention

VS

Hindsight Coalesced (Fingerprinted)

Outbound Calls:	14 calls (-76.7%)
P99 Latency:	39.2 ms
Coalesced:	46 calls
Retained Records:	14 records
I/O Reduction:	-76.7% I/O Overhead